

Project 1: PGP Encryption Report

Executive Summary: This project implements an automated file-encryption service using Pretty Good Privacy (PGP) for securing sensitive report files. It monitors an “incoming” directory for new CSV files, encrypts them using the recipient’s public key (via BouncyCastle/OpenPGP), and writes the encrypted files to an “outgoing” directory. Only one instance in an active-active deployment encrypts any given file (via a “SingleExecutor” cluster handler). The original files are moved to “Processed” or “Error” subfolders after handling. This ensures end-to-end security: PGP protects data at rest and in transit, meeting compliance needs ¹.

Project Overview: The goal is to secure file-based reports by automating their encryption. Stakeholders include security/compliance teams (requiring encryption for audit), operations (handling file workflows), and downstream recipients (who have the private key to decrypt). The scope covers end-to-end encryption of files without manual intervention. Input files (CSV reports) are validated and encrypted, with error handling for bad files. It uses an “active-active” deployment so multiple servers run the service, but coordination ensures only one processes each file ².

Architecture & Components: The system is a Spring-based microservice deployed on multiple nodes in an active-active cluster. Key components: - **File Ingestion:** A directory watcher or scheduled task scans the configured “incoming” path for new CSV files. - **Queue/Handler:** A `SingleExecutorServer` cluster handler (from the com.montran JAE framework) uses a shared queue to ensure only one instance processes a given file at a time, avoiding duplicate encryption. - **PGP Encryption Service:** The core `PGPEncryptionFileProcessorServer` takes a file, uses `PGPFileUtil` (which uses BouncyCastle libraries) to encrypt it, and outputs a “.pgp” file. - **Error/Processed Folders:** Files are moved to subfolders (“Error” for failures, “Processed” for success). - **Supporting Services:** `GenericParamConstService` supplies configuration (incoming/outgoing paths). Logging and Spring context manage dependencies.

```
flowchart TD
    subgraph Cluster
        S1[Service Node A] --- S2[Service Node B]
    end
    S1 -->|Monitor & Encrypt| F[Incoming Files dir]
    S1 --> P[Outgoing (Encrypted) files dir]
    S1 --> E[Error directory]
    S2 -->|Monitor & Encrypt| F
    S2 --> P
    S2 --> E
```

Citations: Active-active deployments share workload across nodes with no downtime ². The encryption uses PGP, which “scrambles the file with a one-time symmetric key and then encrypts that key with the recipient’s public key” ¹. BouncyCastle (the library used) implements the OpenPGP standard for Java ³

⁴.

Technologies & Libraries: Java/Spring Boot for the service. Key libraries: BouncyCastle (`bcprov` , `bcpg`) for PGP encryption/decryption, Java NIO (`java.nio.file`) for filesystem operations, and possibly Spring Data/ORM if any metadata is stored. The code uses Spring annotations (`@Value` , beans), the Montran JAE framework (`SingleExecutorServer` , `SocketRequest`), and common utilities (`StringUtils` , logging). PGPFFileUtil uses `org.bouncycastle.openpgp` classes.

Data Models & Schemas: The system is largely file-based. If persisted, key entities might include: - **FileTask:** (File ID, FileName, Status {Pending, Processing, Done, Error}, Path, Timestamps). - **ConfigurationParam:** (Key, Value) for `PGP_ENCRYPTION_INCOMING_FILE_PATH` , etc. - **Logging:** Messages include file names and outcomes (no formal DB schema shown).

Entity	Key Fields	Description
FileTask	<code>taskId</code> (PK), <code>fileName</code> , <code>status</code> , <code>directory</code> , <code>createdAt</code> , <code>processedAt</code> , <code>errorMsg</code>	Tracks each file's processing state.
Config	<code>paramKey</code> (PK), <code>paramValue</code>	Stores configurable paths/values.

Key Algorithms & Workflows:

1. **Initialization:** Read config for incoming/outgoing paths via `GenericParamConstService` .
2. **Scanning:** Periodically list files in the incoming directory (Java NIO `Files.list`). Filter to only regular files, CSV extension.
3. **Validation:** For each file: check readability; if unreadable or wrong extension, move to `Error/` subfolder (creating it if needed).
4. **Queueing:** Valid files are returned to the queue list for processing.
5. **Processing (service method):** For each queued file: - Verify file still exists. - Ensure the outgoing directory exists. - Construct output file path (`<filename>.pgp`). - Call `PGPFFileUtil.encryptFile(inputPath, outputPath, true)` to perform PGP encryption (ASCII-armored). This does: generate a random session key, encrypt the file data symmetrically (e.g. CAST5/AES), then wrap that session key with the configured public key ¹ . - Log success. Move the original file to `Processed/` . If an error occurs (exception), log error and move file to `Error/` .
6. **File Moving:** Utility method moves files atomically (`Files.move` with `REPLACE_EXISTING`), ensuring no collisions.

This workflow ensures *exactly-once* processing: once encrypted, the original is moved out of the incoming folder.

Deployment & CI/CD: The service is deployed as a Spring Boot JAR on application servers (could be containerized). It runs on multiple nodes (active-active) behind a load balancer or cluster manager, but file scanning likely points to a shared filesystem or NAS. CI/CD could use Git/GitHub, Jenkins or GitLab CI to compile, run tests, and deploy. For example, a pipeline might build the Maven project, run unit tests, and then deploy to test/prod servers via Docker or JVM directly.

Testing Strategy: - **Unit Tests:** For `PGPFFileUtil` , provide sample keys and files to test encryption/decryption (using BouncyCastle's test utilities or GnuPG test keys). Use JUnit/Mockito for classes: mock file I/O to test the handler logic (e.g. simulate unreadable file, invalid extension, successful move). - **Integration Tests:** Actually run the full service on a test folder, drop sample CSVs, and verify encrypted output. Could

automate this on a CI environment. - **Edge Cases:** Test cases for corrupt public key, disk I/O errors (simulate by mocking `Files.move()`), empty incoming directory, simultaneous file drops (concurrency). - **Performance Testing:** (Optional) feed many large files to assess throughput and resource use.

Performance & Security Considerations:

- **Scalability:** File scanning is I/O-bound; ensure the directory listing is efficient (using `Files.list` avoids recursion). For high volume, consider batching or multiple threads, but ensure the `SingleExecutorServer` cluster lock still prevents duplication. - **Security:** PGP encrypts files using best-practice symmetric ciphers (e.g. AES256) combined with asymmetric wrapping ¹. The private key stays on the recipient's side; the service only has the public key. Store keys securely (outside source code, e.g. secured config or secrets manager). Audit logs are necessary so that any file moves and errors are tracked. Avoid leaving plaintext backups. - **Fault Tolerance:** Active-active setup provides high availability ². If one node fails, others pick up scanning. The `SingleExecutor` framework ensures no duplicate work (critical for idempotency).

Challenges & Solutions:

- *Cluster Coordination:* Ensuring only one node processes a file. Solved by `SingleExecutorServer` which checks cluster state (`hasExecuteAccess()`). This avoids race conditions.
- *File Locking:* Possible issues if a file is written concurrently. Solution: perhaps use filesystem locks or conventions (e.g. drop files via temp name, then rename).
- *Large Files:* Encryption is CPU-intensive. For very large files, memory could be an issue if not streaming. The code writes via streams, not buffering entire file in memory (using `PGPUtil.writeLiteralData` with streams).
- *Error Handling:* A file might fail to encrypt (corrupted or wrong format). We catch exceptions, log them, and move files to an `Error/` folder for later inspection.
- *Active-Active Pitfalls:* Race conditions if cluster coordination fails. Rigorous testing in multi-node environments is needed.

Trade-offs & Alternatives:

- Used Java/BouncyCastle vs external PGP command-line: Using the library allows tight integration and avoids shelling out (security + portability).
- File-based workflow vs API-driven: A file drop approach is simpler for batch jobs, but a service with a REST API could offer on-demand encryption. Files simplify integration with legacy systems.
- Active-active vs. Active-passive: Chosen active-active for zero-downtime and load distribution ². A passive standby would have lower resource use but slower failover.
- ASCII-armored output (`.asc`) vs binary `.pgp`: ASCII makes files text-readable but slightly larger. We choose armored output (the code uses `ArmoredOutputStream`) for compatibility with systems expecting `.asc`.

Likely Interview Q&A:

- **Q:** Explain how PGP file encryption works.

A: PGP uses hybrid cryptography ¹. It generates a random symmetric session key to encrypt the file data (e.g. with AES), then encrypts that session key with the recipient's public key. Only the holder of the corresponding private key can decrypt the session key and thus the file. Optionally, the sender can sign the

file to ensure integrity. This two-layer scheme combines performance (symmetric) with secure key distribution (asymmetric) ¹.

• **Q:** *What libraries are used for PGP in Java?*

A: Commonly, BouncyCastle is used. Its `bcpg` and `bcprov` modules implement OpenPGP. We added BouncyCastle as a JCE provider so we can use classes like `PGPEncryptedDataGenerator` and `PGPPublicKeyEncryptedData` to perform encryption ¹.

• **Q:** *How does active-active differ from active-passive, and why use it?*

A: In active-active, all nodes are operational, sharing workload and providing high availability ². If one fails, others continue. Active-passive has standby nodes inactive until a failover, which can incur downtime. We use active-active to maximize uptime and load balance our file-processing jobs ².

• **Q:** *How do you ensure only one node processes a file?*

A: The system uses a cluster-aware `SingleExecutorServer` handler (likely backed by a shared database or lock service). Before a node processes a file, it calls `hasExecuteAccess()`, which checks if any other node is handling it. This prevents duplicate processing.

• **Q:** *What happens if encryption fails for a file?*

A: The code catches exceptions and moves the original file into an `Error/` subfolder (creating it if needed). It logs the error for diagnosis. This prevents failed files from blocking the queue.

• **Q:** *How would you test the encryption functionality?*

A: Unit-test `PGPFileUtil`: generate a test key pair (or use a known test key), encrypt a small file, then decrypt with the private key and verify content. Integration test the whole flow by dropping a sample file in "incoming" and verifying the encrypted output and correct file moves.

• **Q:** *Security considerations?*

A: Secure the key (private key never on this service, public key is safe to distribute). Ensure encrypted files cannot be tampered with (PGP encryption often includes integrity protection). Log access and moves. Encrypting data at rest in compliance-sensitive environments adds a layer beyond TLS (since TLS only protects in motion) ⁵.

Cheat Sheet (PGP Encryption Report): *[One-page bullet summary]*

- **Purpose:** Automate PGP encryption of CSV reports for secure file transfers.
- **Process:** Monitor incoming dir → validate CSV files → encrypt each with PGP → output `.pgp` in outgoing dir → move original to `Processed/` or `Error/`.
- **Key Tech:** Java (Spring Boot), BouncyCastle (OpenPGP), Montran JAE framework (`SingleExecutorServer`), Java NIO for file I/O.
- **Security:** Uses hybrid PGP encryption (symmetric AES + RSA). Public key from config, private key held by recipient. Ensures files remain encrypted at rest.
- **HA Setup:** Active-active cluster (multiple nodes) with single-executor lock to avoid double-processing ². No downtime on server failure.
- **Error Handling:** Unreadable/invalid files → move to `Error/`. Exceptions → log + move to `Error/`.

- **Config:** Paths are externalized (`PGP_ENCRYPTION_INCOMING_FILE_PATH` , etc.). Supports `.csv` only (configurable).
- **Performance:** Stream-based I/O. Avoid reading entire file into memory. Do encryption off-peak if heavy.
- **Testing:** Unit-test PGP util (encrypt/decrypt round-trip). Integration: drop sample files, verify outcomes.
- **Interview Tips:** Explain PGP flow (two-layer encryption) ¹ , active-active benefits ² , error/file handling logic.

5-Minute Pitch:

“Problem: We needed a reliable way to encrypt sensitive report files for downstream processing, with minimal human intervention. **Solution:** We built an automated PGP encryption service. It watches a designated ‘incoming’ folder; whenever a new CSV appears, it encrypts the file using our public key and outputs it to an ‘outgoing’ directory. The original is then archived or errored out. This uses the BouncyCastle OpenPGP library, so files are protected both at rest and in transit by the industry-standard hybrid encryption mechanism ¹ . We deployed it in an active-active cluster, so multiple nodes run the service but coordination ensures only one handles each file ² . **Benefits:** This ensures compliance (files stay encrypted end-to-end), removes manual steps, and provides high availability. If a server goes down, others continue processing without interruption. For interviewers, focus on how you handle key management (storing public key in config, never exposing private keys), how PGP uses a combination of symmetric and asymmetric encryption ¹ , and how we ensure exactly-once processing with our cluster design. The toughest challenge was coordinating between nodes, which we solved with a clustered executor. Overall, the service achieves secure, automated file encryption at scale.”

Project 2: Bulk UPI QR Code Generator

Executive Summary: This project automates the generation of QR codes in bulk for UPI (Unified Payments Interface) transactions. It reads input records (e.g. from a CSV file) containing merchant/payment parameters and outputs a ZIP archive of QR code images and a result report. The service supports both **static** QR codes (fixed for a merchant) and **dynamic** QR codes (unique per transaction, with amount and reference encoded). For each input row, it validates fields (merchant ID, VPA, amount, expiry, etc.), calls a QR generation service, saves the QR image, and logs success or detailed errors. At the end, all QR images and a response CSV are zipped for download. This accelerates onboarding of merchants by programmatically handling thousands of QR codes, ensuring adherence to UPI standards.

Project Overview: We aim to streamline merchant QR code provisioning for digital payments. The scope covers file parsing, QR creation, packaging, and status reporting. Stakeholders include product teams offering UPI integration, merchant onboarding, and finance operations. The tool takes as input a list of records (containing fields like merchant ID, Virtual Payment Address (VPA), amounts, etc.) and outputs a downloadable ZIP with JPEG QR images and a summary response file. It integrates with existing UPI backend services (e.g. `intentQRService`) that encapsulate UPI QR rules and signing. The service handles two modes: - **Static QR Codes:** Pre-generated codes that merchants can reuse. They contain fixed VPA and merchant info. ⁶

- **Dynamic QR Codes:** Generated per transaction, embedding amount and transaction details, providing higher security (each code is unique) ⁷ .

Architecture & Components: The system is a backend batch service (Java/Spring). Components: - **Input Handler:** Receives a `FileBean` referencing the uploaded CSV. Determines base directories for output (e.g. `/Download`, `/Response`). - **Parser/Validator:** Splits each CSV row into fields. Validates business rules: correct VPA (using `upiAcquirerApiValidator`), terminal permissions, numeric amounts, expiry dates, custom QR reference prefixes, etc. Builds `QRIntentURLParams` data objects. - **QR Service:** Calls `IntentQRService` to generate QR data (via `getQrImageSrc`). For dynamic mode, may create an `IntentQR` object to record the transaction with `intentQRService.save()`, sign the data if in secure mode, etc. - **Image Processor:** Decodes the Base64 QR image into bytes, writes to a JPG file named e.g. `{rowIndex}_{VPA}.jpg` in an `images/` folder. Supports branding logos if enabled. - **Zip Packaging:** Collects all generated images and the response CSV into one or more ZIP files (splitting into multiple 4MB chunks if needed via `createZipFiles`). Uses Java `ZipOutputStream`. Ensures duplicate image names are handled (skipped). - **Response Report:** Builds a CSV or text file of responses, with headers like `MerchantId`, `VPA`, `Amount`, `Result`. Populates each line with success or error message. - **Cleanup:** Deletes individual files after zipping, updates `FileBean` record with final ZIP path and status counts. - **Error Handling:** Logs errors per row; unsuccessful rows still go into the response file with an "Error:" prefix.

```

flowchart LR
    subgraph Batch_Job [Batch Job]
        A[Input CSV File] --> B[Row Parser/Validator]
        B -->|valid static| C[generateStaticQR]
        B -->|valid dynamic| D[generateDynamicQR]
        C & D --> E[Collect Base64 QR Images]
        E --> F[Write JPEG files to images folder]
        F --> G[Zip images + report.csv]
    end
    subgraph Output
        G --> H[ZIP archive of QR images and report]
    end

```

Citations: UPI static QR codes embed fixed merchant info and stay the same each transaction ⁶, while dynamic QR codes are generated per transaction (changing code each time) for added security ⁸. This project implements both modes as per business need.

Technologies & Libraries: Java/Spring backend. Likely uses Spring MVC or batch runner. Key libraries: Java's built-in CSV/IO utilities (or Spring's file utilities) for parsing; `java.util.zip` for zipping; Base64 (Java 8's `Base64`); image I/O (writing JPG); and calls into internal services: - `intentQRService`: abstracts UPI QR creation (likely provided by the financial platform). - `upiAcquirerApiValidator`, `upiMerchantTerminalBOServiceImpl`, `upiAcquirerApiValidator`: for UPI-specific data (merchant, terminal info). - Database access (via Spring Data or MyBatis) for retrieving merchant/terminal/VPA data if needed (code hints at services like `getTerminalByVpaPattern`). - Logging via `DefaultLogger`. We use `Paths` and `Files` (Java NIO) for filesystem operations. No UI components needed – it's backend-only.

Data Models & Schemas: Key entities (simplified):

Entity	Fields	Description
FileBean	fileId (PK), fileName, directory, totalItems, totalItemsComputed, status	Tracks each batch job's file; final ZIP location saved here.
QRInputRecord	rowIndex, clientId, virAddr (VPA), merchantName, minAmount, amount, qrRefId, qrNote, qrExpire	Represents one input line of QR parameters.
FileFormatBean	(format metadata, e.g. separators, header presence)	Configuration for parsing columns.
IntentQR / QRIntent (UI model)	intentLink (data string), data (binary), and parameters (ver, mode, pa, pn, mc, tid, mid, mam, tr, tn, QRexpire)	Used internally by IntentQRService to model a QR payment.

The actual `intentQRService` likely maps its parameters to the standardized UPI QR payload as per NPCI's specification.

Key Algorithms & Workflows:

- 1. Read Input:** Load all lines from the uploaded CSV (`allRows`). Initialize counters and output lists.
- Iterate Rows:** For each `rowString`:
 - Split by separator (`,`) into fields.
 - Determine mode: static or dynamic based on a flag (`isDynamic`).
 - Extract fields into variables (`clientId`, `virAddr`, `vpaName`, `minAmt`, `amtValue`, `qrRefId`, `qrNote`, `qrExpire`).
 - **Validate Fields:** Check VPA syntax and existence (`upiAcquirerApiValidator.getUpiVpa`), or resolve via merchant terminal if VPA not directly found. Check that `qrMedium` is allowed for this terminal. Verify numerical fields are valid decimals and in range ($\text{min} \leq \text{amount}$). Verify date parse for expiry not in past. Validate `qrRefId` formats (prefixes, numeric parts). Accumulate any error messages in a `StringBuffer`.
 - **Generate QR:**
 - For *static mode*, call `getStaticMerchantQR(...)`: builds a `QRIntentURLParams` object with fields `ver="01"`, `pa` (VPA), `pn` (merchant name), `mc` (MCC), `mid`, `mtid`, `qrMedium`, `mode=02`, `TR`, `TN` (optional). It then calls `intentQRService.getQrImageSrc(500,500, qrData)`, which returns a data URL (e.g. `"data:image/png;base64,..."`). If branding is on, calls `generateQRImageSrc` to overlay logo. It extracts the Base64 part and writes to a JPG file path `imagesFolder/{row}_{VPA}.jpg`.
 - For *dynamic mode*, similar but also considers `amount`, `minimumAmount`, `qrExpire`, and `note`. It may create a new `IntentQR` record via `intentQRService.newIntentQR(...)`, set expiry and other dynamic parameters, sign if needed, and save it (persisting a QR code for the transaction). Then get the image as above.
 - **Record Result:** If an image was generated (`filePath != null`), add path to `validQRPaths` and append to `allRecords` the original row plus the Base64 string for "QR" column; increment `successCount`. Otherwise, append "Error: <message>" to `allRecords` and increment `failureCount`.
- Create Directories:** Ensure folders exist (`imagesFolder = /Download/<fileName>/` and a `zip` subfolder).
- Generate ZIP(s):** Call `createZipFiles(validQRPaths, imagesFolder, fileBean)` which splits images into one or more ZIP files not exceeding ~4MB each. This returns list of ZIP paths.
- Generate Response File:** Build a CSV/Excel at `imagesFolder` with columns (Merchant Id, Terminal Id, VPA, ..., Response). Each row from `allRecords` is written.
- Finalize ZIP:** Add the response file to the ZIP list. Create a final ZIP stream, iterating all files (images + response CSV). Avoid duplicate entries via a set.
- Cleanup:** Delete individual image files and the response file after zipping. Update `fileBean` to point to the final ZIP (`directory` and `fileName` updated) and set totals (`totalItems`,

`totalItemsComputed`). 8. **Error Handling:** All steps are wrapped in try/catch. On exception, log an error and return false (so the job can be marked failed).

This pipeline converts raw input data into a user-friendly deliverable: a ZIP containing QR code images and an outcomes report.

Deployment & CI/CD: Similar to Project 1: Java application deployed (could be triggered on file upload or via a scheduler). Likely runs as a background worker or a Spring Batch job. CI pipeline builds the code, runs automated tests (unit tests on parsing and zipping logic, integration test with a sample CSV), and deploys to servers. If containerized, use Docker and Kubernetes or similar for scaling. Since processing can be heavy (many images), we might allocate sufficient memory/CPU or use parallelism cautiously.

Testing Strategy:

- **Unit Tests:** Validate parsing logic (correct splitting, trimming). Test `getStaticMerchantQR` and `getDynamicMerchantQR` using mocked services (`upiAcquirerApiValidator`, `intentQRService`) to simulate valid and invalid inputs. For example, given a valid VPA and amount, assert that a non-null path is returned; for invalid data, assert null and correct error message.
- **Integration Tests:** End-to-end: feed a test CSV with known merchant data and check that the ZIP output contains the expected number of images and a correct CSV. Use small images or a mocked `intentQRService` that returns a fixed base64.
- **Edge Cases:** Large numbers of rows (performance), duplicate VPAs (should generate distinct filenames), reserved keywords or malicious input (ensure SQL injection is not possible when looking up terminals), invalid image data (should log and continue).
- **Visual Inspection:** Manually scan a generated QR with a UPI app to verify it encodes the correct payload (using a stable base64 decoding).

Performance & Security Considerations:

- **Memory:** Each QR image is base64-decoded into bytes (in memory) and written to disk. For many large images, ensure JVM heap can handle it, or generate and write streams directly. We clean up images after zipping to free space.
- **Large ZIP Handling:** We split output into multiple ZIPs if >4MB (the code's `MAX_ZIP_SIZE`). This prevents huge single archives that might exceed client limits.
- **Concurrency:** If multiple files are processed simultaneously (from different users), ensure separate `FileBean` contexts and unique temp folders to avoid collision.
- **Security:** QR data includes sensitive transaction info. We use secure coding: validate all input fields thoroughly (as seen in code) to prevent injection of bad data. The `intentQRService` likely signs the QR payload when in secure mode, protecting integrity. If static QR is just a VPA, it's safe to reuse. Ensure the ZIP is only accessible to authorized requesters. No direct user-supplied code execution is involved.
- **Error Reporting:** The response CSV shows any invalid input (e.g. "Invalid VPA"); by not halting the entire batch on one bad line, we provide graceful degradation.

Challenges & Solutions:

- **Variable Input Formats:** The code supports dynamic vs static modes and different numbers of columns. The service `fileFormatB0Service.getRowCells()` handles splitting. We ensure robust parsing by trimming and checking array bounds.
- **Error Isolation:** One bad record shouldn't stop the whole job. The implementation catches exceptions per-row and logs an error in that row's result `6` `8`.
- **Duplicate Entries in ZIP:** If two images had the same relative path, the zip loop could overwrite. We track added entries (`addedEntries` set) to skip duplicates.

- *Large Files/Archives*: To avoid huge zips, `createZipFiles` partitions files across multiple zip outputs when a size threshold is reached. This prevents memory issues and client download limits.
- *Thread Safety*: The code seems single-threaded per job. If we parallelized rows, we'd need to synchronize on writing to shared resources. Keeping it sequential simplifies this.

Trade-offs & Alternatives:

- Chose file+zip output vs streaming API: This batch approach is user-friendly for large lists but not real-time. An alternative could be a REST API generating one QR at a time.
- Base64 in-memory vs direct image generation: We get a base64 string from the service, decode it, and write. We could also use libraries to render QR from data directly (e.g. ZXing), but the existing `intentQRService` ensures compliance with UPI specs (signatures, payload format).
- Static vs Dynamic mode: Static QR is easier (no expiration or amount), but not suitable for large or variable payments. Dynamic adds complexity (transaction record, expiry, amounts) but improves accuracy ⁸. We support both as needed.
- Multiple ZIPs vs one: Multi-zip increases complexity but avoids over-sized archives. We could also stream files for download, but zipper is simpler for consumers.

Likely Interview Q&A:

Q: *What is the difference between static and dynamic UPI QR codes?*

A: Static QR codes encode fixed merchant information (VPA) and remain the same each time ⁶. The payer can enter the amount manually. Dynamic QR codes are unique per transaction: they include the specific amount, order info, and change each time ⁸. Dynamic codes reduce errors and fraud since each code is tied to one transaction with an encoded ID.

• **Q:** *How does the service generate a QR image?*

A: We construct a string of QR parameters (`QRIntentURLParams`) per the NPCI UPI spec (fields like MID, TID, PA, PN, MC, AM, TR, etc.). Then we call `intentQRService.getQrImageSrc(500, 500, qrData)` which returns a PNG as a Base64 data URL. We strip off the prefix and decode the Base64 into bytes, then write to a JPEG file. If merchant branding is enabled, we post-process the image (via `generateQRImageSrc`) to overlay a logo.

• **Q:** *How do you handle validation?*

A: For each field, we check format and business rules. Example: verify VPA exists (`getUpiVpa` returns a bean), or else lookup by pattern. Check merchant terminal is active and supports the requested QR medium. Validate that amounts are decimals and within allowed ranges. Confirm date strings parse to valid future datetimes. Any failure appends an error message (e.g. "Invalid QR Expire") and that row is marked failed.

• **Q:** *Why use ZIP files here?*

A: Bulk operations often produce many images, which are inconvenient to download individually. Packaging them into a single archive (with a summary CSV) is user-friendly. We also enforce a 4MB size limit per zip (as per requirements) to avoid huge single downloads, splitting into multiple zip files if needed.

• **Q:** *How to ensure no duplicate files in ZIP?*

A: The code uses a `Set<String>` to track which ZIP entry names have been added. If an entry is

already present, we skip adding that file again. This ensures unique paths and avoids exceptions in the zip stream.

- **Q:** *What if the number of records is very large?*

A: We might stream-process rows and images to avoid holding everything in memory. Currently we build lists of paths and responses, but we could adapt it to a streaming model. Zipping is done entry-by-entry, which helps memory. The splitting logic (`createZipFiles`) also prevents any one file from growing too big. For extremely large loads, we could parallelize row processing or scale-out multiple batch jobs.

- **Q:** *What testing would you do?*

A: Unit tests for parsing and validation (e.g. valid vs invalid VPAs and amounts). Integration test with a small sample CSV and compare expected outputs (count, file names, contents). Possibly end-to-end test by scanning a generated QR code to confirm it encodes the right UPI payload (on a UPI app).

- **Q:** *What are security concerns?*

A: QR codes contain payment info. We must ensure the data string (`qrData`) is correctly signed in secure mode (the service does signing via `intentQRService.isSecureMode` check). Also, avoid injection attacks: since `qrData` is generated from structured parameters, we don't concatenate raw input into queries. Access control ensures only authorized users can trigger this job and download the results.

Cheat Sheet (Bulk QR Download): *[One-page bullet summary]*

- **Purpose:** Generate UPI QR code images in bulk from input data (CSV), for both static (fixed) and dynamic (transaction-specific) QR codes ⁶ ⁸ .
- **Process:** Parse each row → validate fields (VPA, merchant, amounts, expiry) → build QR parameters (MID, TID, PA, etc.) → call QR generation service → decode Base64 QR image → save image file → record success or error in result list.
- **Static vs Dynamic:** Static QR = reusable fixed code; Dynamic QR = unique per transaction with embedded amount/ID ⁸ .
- **Config & Inputs:** Input: `FileBean` with base directory, file name. Output: `fileBean.totalItems` , `totalItemsComputed` , updated with ZIP path (`*.zip`). Separators and formats driven by `FileFormatBean` .
- **Output:** One or more ZIP archives under `<Download>/<fileName>.zip` , containing: JPEG images of QR codes and a response CSV with results.
- **Validations:**
 - Check VPA/merchant exists (`upiAcquirerApiValidator.getUpiVpa` or merchant DB).
 - Validate numeric fields (`isDecimalNum`), amounts ($\text{min} \leq \text{amt}$, $\text{amt} > 0$).
 - Check transaction IDs (`qrRefId`) follow expected prefixes and formats.
 - Validate dates (Expiry must be after now).
- **Error Handling:** Each row's errors are collected and written next to it in the response file. Bad rows don't stop the batch.
- **Tech:** Java/Spring, JDBC or JPA for UPI master data, internal UPI QR service (`IntentQRService`), Java NIO for file ops, `java.util.zip` for zipping, Java Base64 and image I/O.

- **Performance:** Stream files; limit ZIP size (~4MB) to avoid huge archives. Clean up temp files after zipping.
- **Interview Tips:** Mention UPI QR specs (NPCI), difference static/dynamic ⁸, validation logic (robust checks), packaging into ZIP for download, and how the system scales to large input. Also note avoiding race conditions in file output and ensuring atomic updates to `FileBean`.

5-Minute Pitch:

“Problem: Our payments platform needed a way to provision QR codes in bulk for merchants. Generating them one-by-one is tedious and error-prone. **Solution:** We built a **Bulk QR Generator** service. Given a spreadsheet of merchant/VPA details (with amounts, expiry, etc.), it programmatically creates either static or dynamic UPI QR codes for each entry ⁶ ⁸. It validates all inputs (ensuring each VPA and merchant ID exists, amounts are valid, etc.), then constructs the proper UPI payload and calls our `IntentQRService` to generate the QR. Each QR image (PNG) is decoded and saved as a JPG. We collect all images and the processing results into one or more ZIP files, which the merchant can easily download. **Impact:** This automates what used to be a manual, repetitive process, saving time and reducing mistakes. Dynamic QR codes include transaction references and amounts for security ⁸, while static codes are simpler for small payments. In interviews, highlight how we handle static vs dynamic modes, how we sanitize inputs and report errors per row, and how we efficiently package results. Mention the splitting logic for large ZIPs and how we ensure no filename collisions. This solution scales to thousands of entries and integrates with our UPI back-end to guarantee compliance and security.”

¹ ⁵ PGP File Encryption: How It Works And Why It Matters

<https://www.jscape.com/blog/how-to-automatically-pgp-encrypt-a-file-upon-upload-using-triggers>

² Active-Active Vs. Active-Passive High-Availability Clustering | JScape

<https://www.jscape.com/blog/active-active-vs-active-passive-high-availability-cluster>

³ ⁴ PGP Encryption and Decryption Using Bouncy Castle | Baeldung

<https://www.baeldung.com/java-bouncy-castle-pgp-encryption>

⁶ ⁷ ⁸ Understanding UPI QR Codes: Static vs. Dynamic

<https://paytm.com/blog/payments/upi/are-there-different-types-of-upi-qr-codes/>